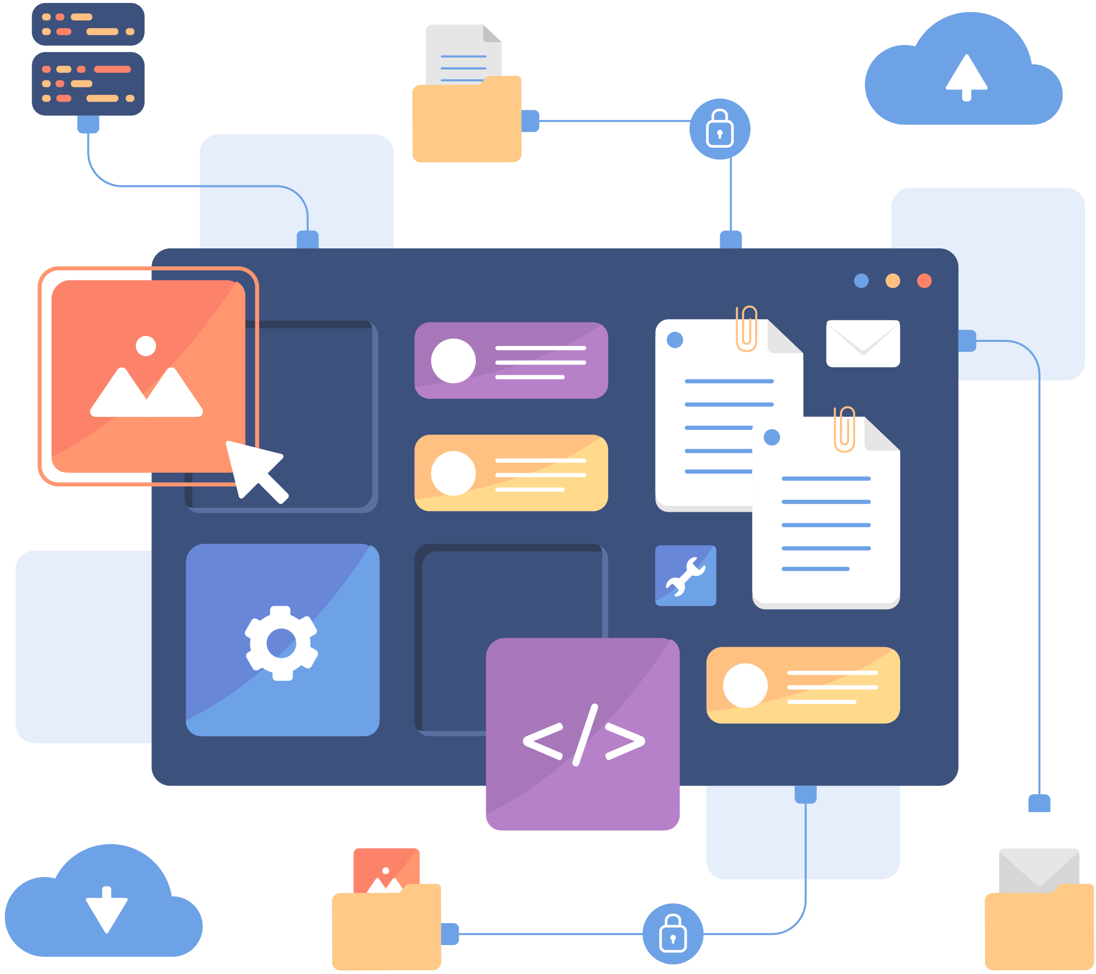
Image Generative Models

Group	42
XIANG Bojie	25400444
ZHOU Tianchi	25409638
ZHU Tianxiao	25407732
SU Zinao	25434675
ZHAO Zihao	25418823



INTRODUCTION

INTRODUCTION

1

VAE

Synthesize handwritten characters

2

GAN

Convert digits and letters to realistic handwritten images.

3

DDPM

Generate class-conditional natural images

Dataset and preprocessing

Dataset	Preprocessing
Handwritten dataset from Hugging Face	Split 70,000+ training set images into new training/validation sets at a 9:1 ratio
Cifar-10	Using in its original form.

https://huggingface.co/datasets/PranomVignesh/handwritten-characters-complete/tree/main
https://www.cs.toronto.edu/~kriz/cifar.html

VAE

VAE

Our VAE model consists of an Encoder and a Decoder. The encoder performs downsampling through convolution, and the decoder applies transposed convolution for upsampling.

Encoder is connected to two Linear modules that predict the **mean(u), variance** respectively.

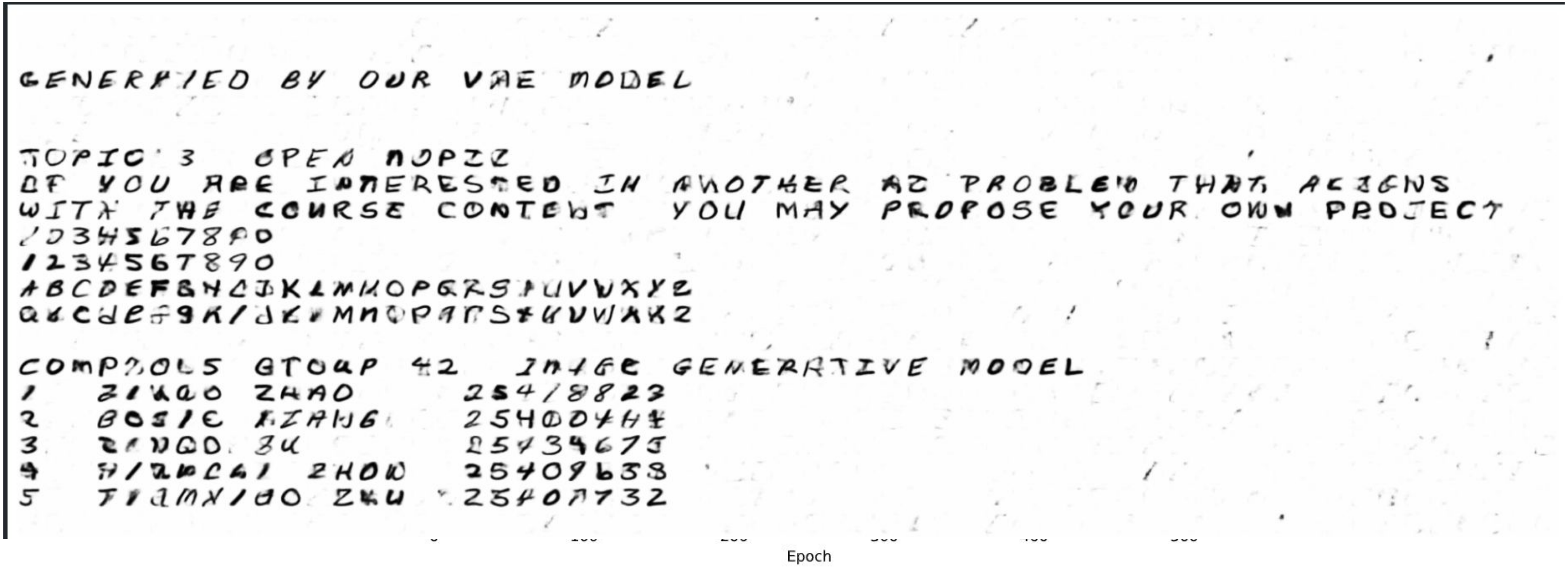
Reparameterization is used that ensuring effective backpropagation during training.

z = mean + variance * epsilon   send z to Decoder

Loss function:

Reconstruction Loss + KL Divergence Loss

VAE



Gan

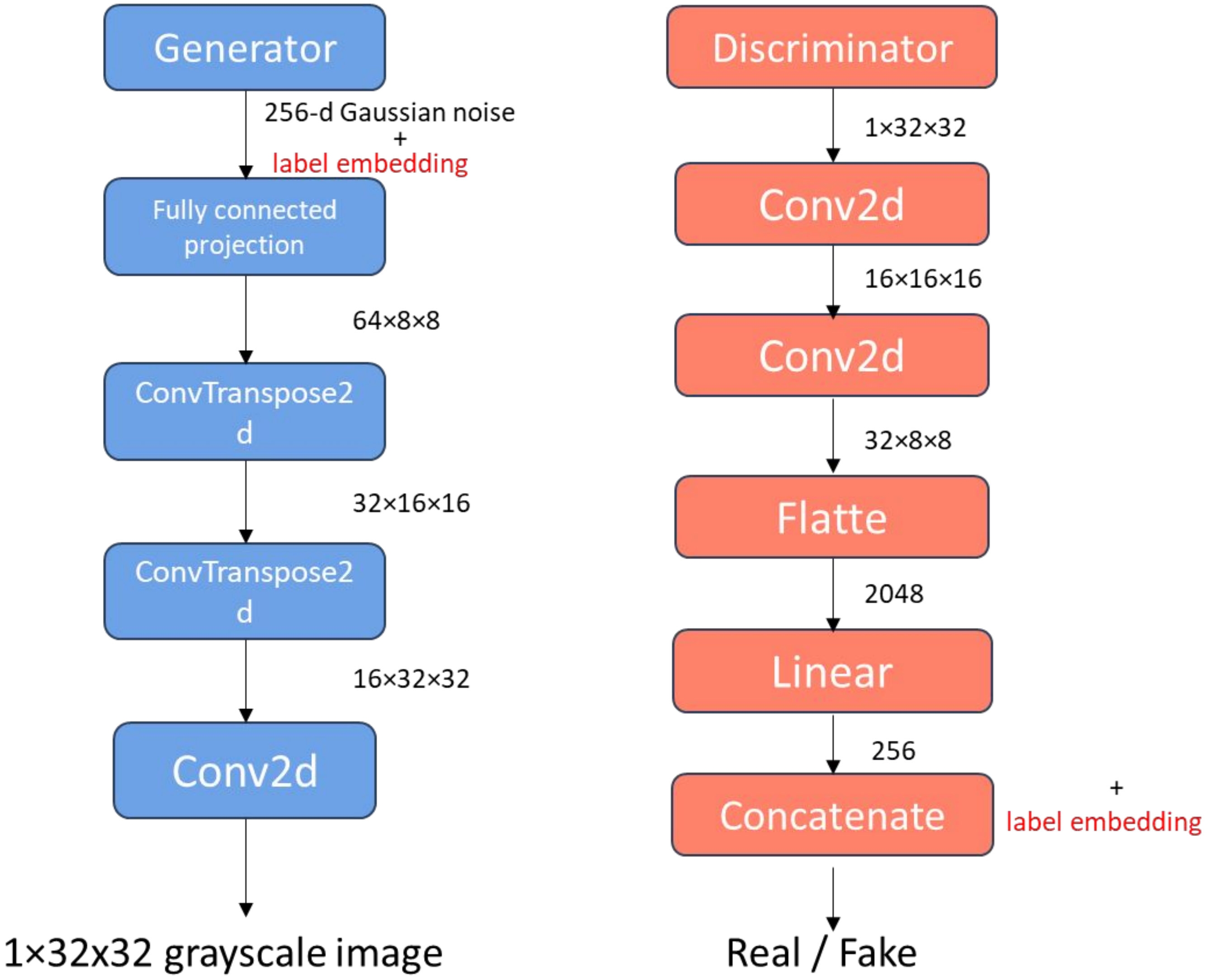
Convolutional GAN Architecture

Generator

- Input: 256-d Gaussian noise + label embedding
- Fully connected projection → 64×8×8 feature map
- Up-sampling via 2 ConvTranspose2d blocks
- Output: 32×32 grayscale image

Discriminator

- Two Conv blocks (downsampling: 32→16→8)
- Flatten → Linear(256)
- Concatenate label embedding
- Binary classification: real / fake



GAN

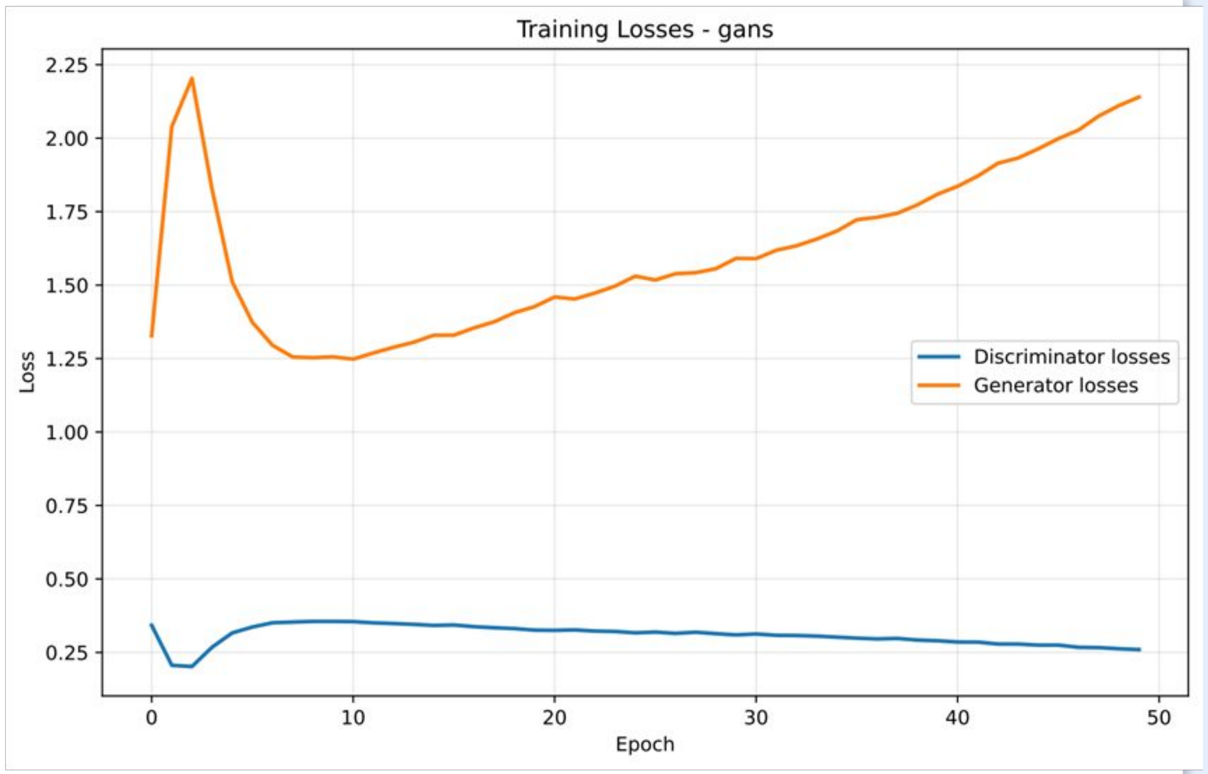
Training Strategy

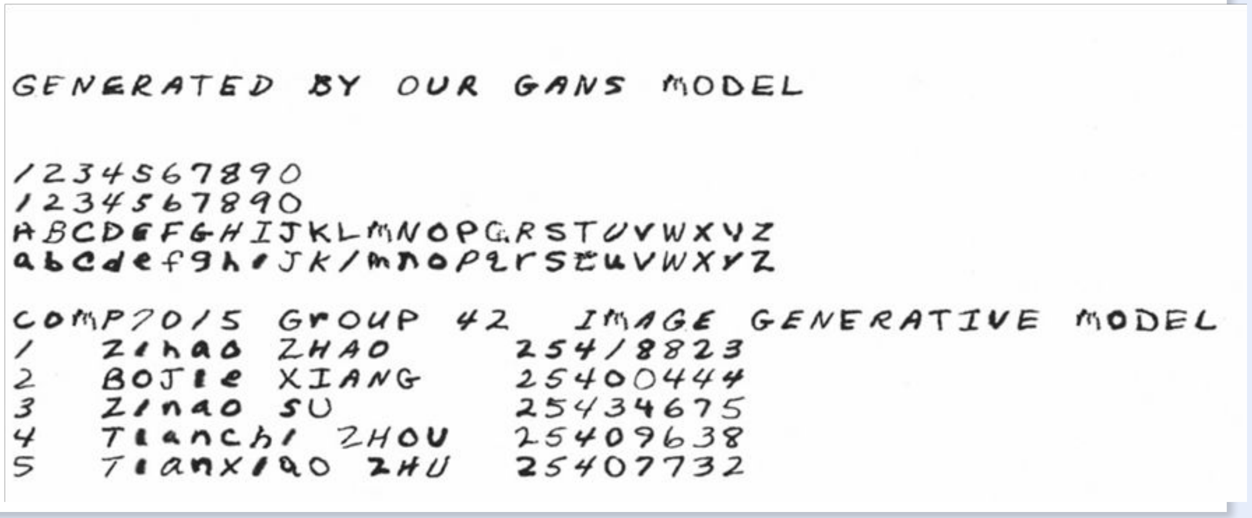
Loss: Standard adversarial loss

Challenge: Discriminator stronger than Generator

Custom balancing strategy:

- When G loss too high → Increase G update steps





Evaluation and Comparison

Evaluation : FID

- Metric/tool: FID with PyTorch
  InceptionV3, pytorch_fid
- Similar architectures of decoder of the VAE and generator of the GAN.

```
if os.path.exists(path1) and os.path.exists(path2):
        score = fid_score.calculate_fid_given_paths(
            [path1,path2],
            device=torch.device('cuda' if torch.cuda.is_available() else 'cpu'),
            batch_size=128,
            dims=2048
        )
```

Result

- Validation FID: GAN 28.14 vs VAE 49.97 (gap 21.83).
- Training FID: GAN 28.12, VAE 50.36 (gap 22.24)
- Val-Train: GAN +0.02, VAE -0.39

Model	Train FID	Validation FID	Val - Train
Conditional VAE	50.36	49.97	-0.39
Conditional GAN	28.12	28.14	+0.02

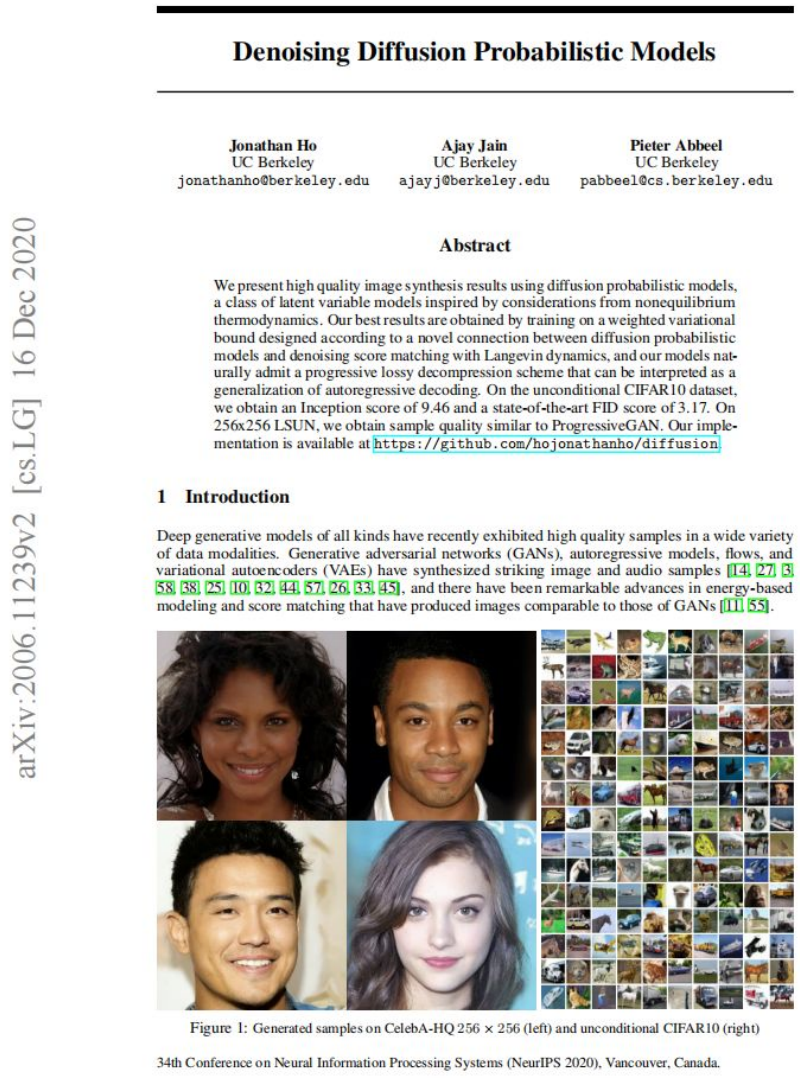
Weakness

- VAE
  - Generates blurry images; lacks sharp details despite stable training
- GAN
  - Unstable training and sensitive to hyperparameter tuning
  - Mode collapse: limited output diversity for some classes

(a) Blurry VAE

(b) Mode Collapse GAN

DDPM



arXiv:2006.11239v2 [cs.LG] 16 Dec 2020

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajay Jain
UC Berkeley
ajayj@berkeley.edu

Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

Abstract

We present high quality image synthesis results using diffusion probabilistic models, a class of latent variable models inspired by considerations from nonequilibrium thermodynamics. Our best results are obtained by training on a weighted variational bound designed according to a novel connection between diffusion probabilistic models and denoising score matching with Langevin dynamics, and our models naturally admit a progressive lossy decompression scheme that can be interpreted as a generalization of autoregressive decoding. On the unconditional CIFAR10 dataset, we obtain an Inception score of 9.46 and a state-of-the-art FID score of 3.17. On 256x256 LSUN, we obtain sample quality similar to ProgressiveGAN. Our implementation is available at https://github.com/hojonathanho/diffusion

1 Introduction

Deep generative models of all kinds have recently exhibited high quality samples in a wide variety of data modalities. Generative adversarial networks (GANs), autoregressive models, flows, and variational autoencoders (VAEs) have synthesized striking image and audio samples [14, 27, 3, 58, 38, 25, 10, 32, 44, 57, 26, 33, 45], and there have been remarkable advances in energy-based modeling and score matching that have produced images comparable to those of GANs [11, 55].

Figure 1: Generated samples on CelebA-HQ 256 × 256 (left) and unconditional CIFAR10 (right)

34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.

We trained larger model based on the DDPM paper.

hyper_parameter beta same as the original paper: 0.0001 -> 0.02   T=1000

$$\alpha_t := 1 - \beta_t \text{ and } \bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s$$

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right) + \sigma_t z$$

UNet model
we are going to train

DDPM add_noise AND de_noise demo

UNet Structure (25m parameters)

time_encoding + label_embeding

encoding_mlp

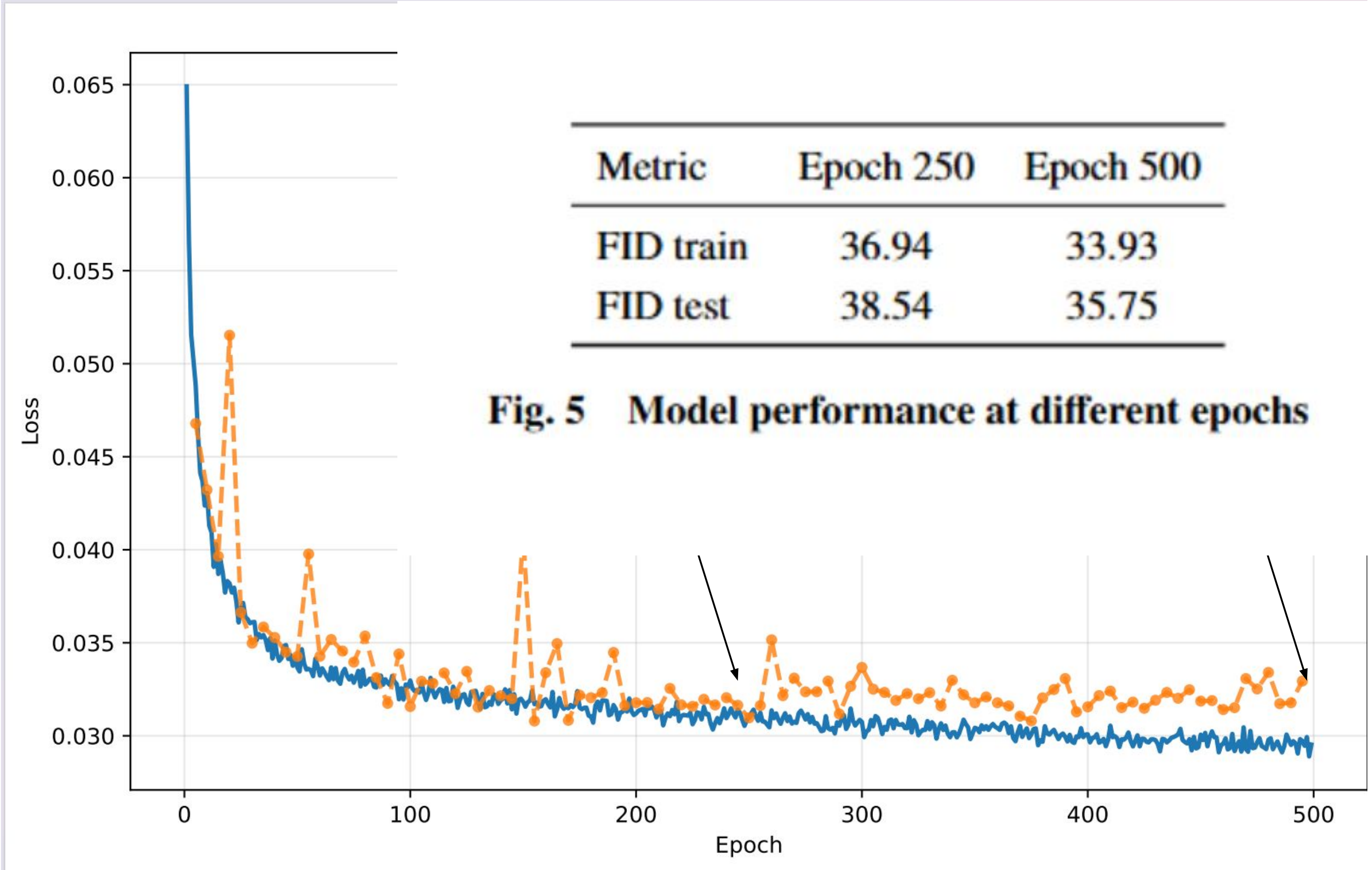
input +

input+

```
in Unet (skip connection) :
        down:
            for _ in 4:
                conv_block(channels -> 2 * channels, residual connection ) followed by self_attention
                max_pooling(2)

        botttom_block ()

        up:
            for _ in 4:
                conv_block(channels  -> channels/2, residual connection ) followed by self_attention
                upsample(2, nearest)

return noise_predict
```

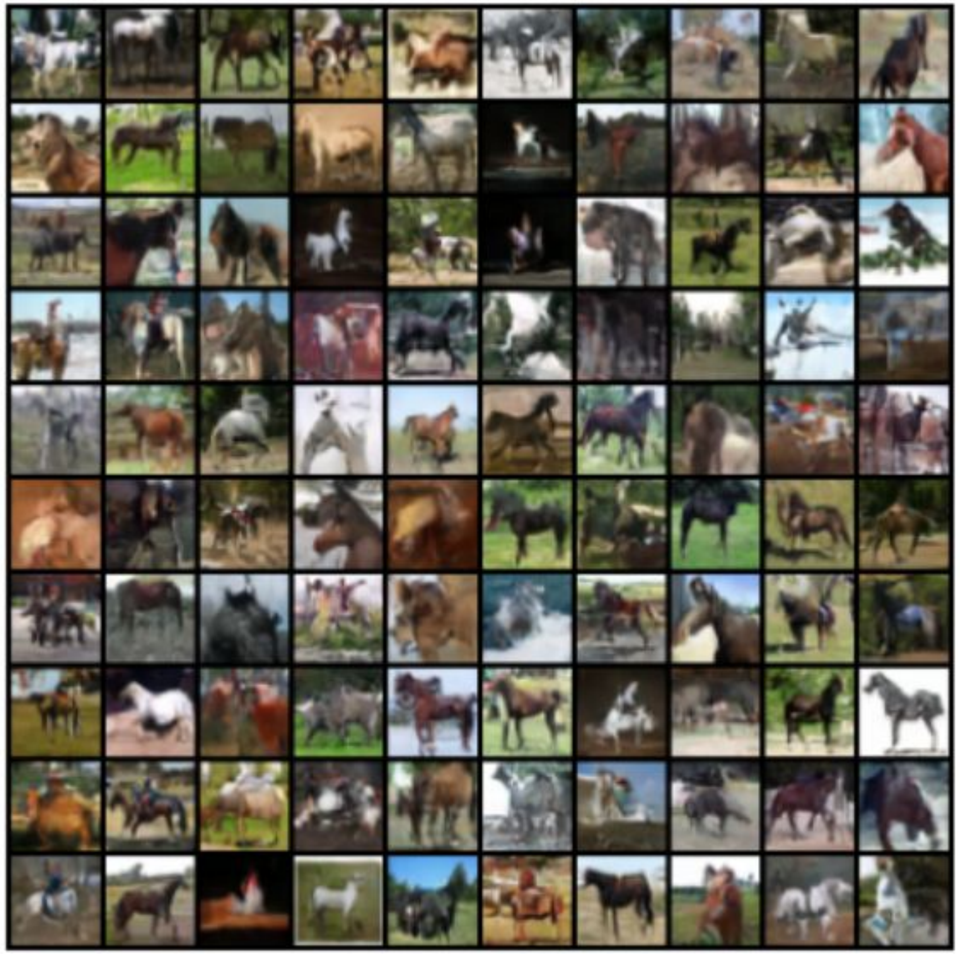
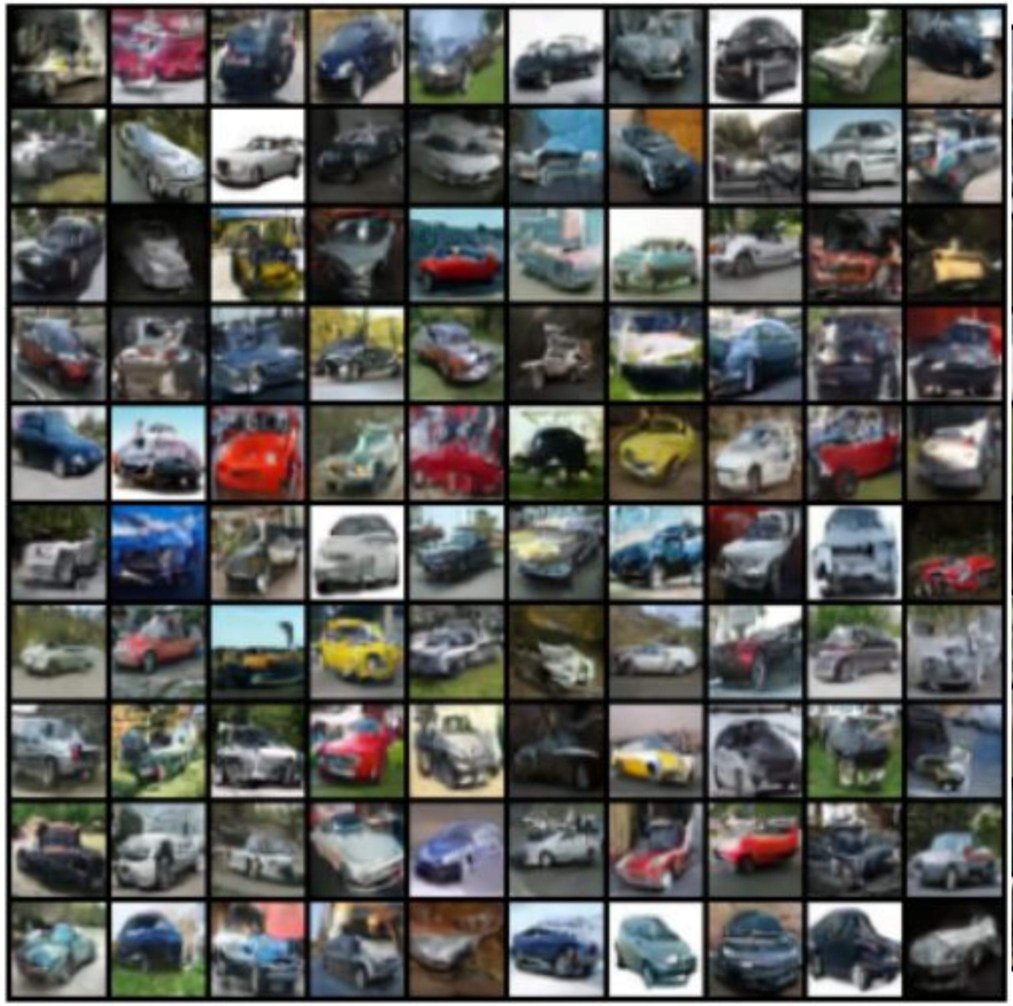
Training

loss = MSE(real_noise, predict_noise)
on the CIFAR-10 dataset normalized to (-1, 1)
batch size = 128
500 epochs (195,500 iterations)
on RTX 4060 (60 s/epoch)

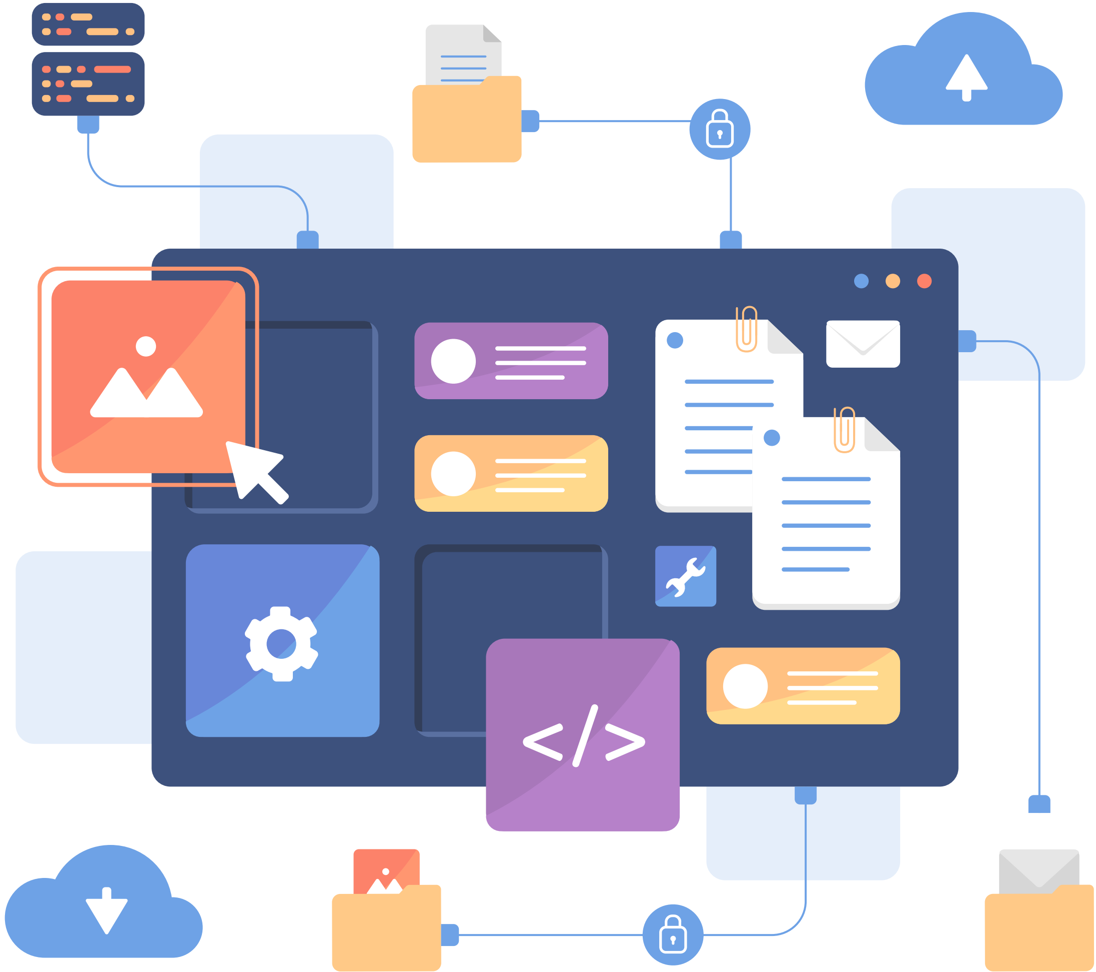


Metric	Epoch 250	Epoch 500
FID train	36.94	33.93
FID test	38.54	35.75

Fig. 5 Model performance at different epochs


THANK YOU



Encoder	Decoder	Conditional Embedding Layer
Convolutional layers: Conv2d layers **Output layers:** 2 Linear layers outputting mean (u) and log variance (log_var) respectively	**Input layer:** 1 Linear layer **Transposed convolutional layers:** ConvTranspose2d layers **Output layer:** Sigmoid	1 Embedding layer